ตราโรงเรียนบางละมุง (วางไฟล์โลโก้)

การประเมินประสิทธิภาพเชิงเปรียบเทียบของระบบจำลองคอมพิวเตอร์ควอนตัมขนาดเล็กบนสถาปัตยกรรมไมโครคอนโทรลเลอร์ที่หลากหลาย สำหรับการประมวลผลที่ขอบข่าย

Comparative Performance Benchmarking of Quantum Micro-Simulators Across Diverse Microcontroller Architectures for Edge Computing

ผู้จัดทำโครงงาน
นายธีรศักดิ์ รักความสุข
ชั้นมัธยมศึกษาปีที่ 6
โรงเรียนบางละมุง จังหวัดชลบุรี

ครูที่ปรึกษาโครงงาน
ผู้ช่วยศาสตราจารย์ ดร.วัชรพัฐ เมตตานันท์
ภาควิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์
มหาวิทยาลัยเกษตรศาสตร์ วิทยาเขตศรีราชา

บทนำ (ที่มาและความสำคัญ)

ระบบ IoT กำลังเปลี่ยนจากการพึ่งพาคลาวด์ไปสู่การประมวลผลที่ขอบข่าย (Edge Computing) เพื่อลด Latency แต่ไมโครคอนโทรลเลอร์แต่ละสถาปัตยกรรมมีข้อจำกัดด้านหน่วยความจำต่างกัน การจำลองวงจรควอนตัมซึ่งต้องการหน่วยความจำเพิ่มแบบเอ็กซ์โพเนนเชียล (2^n ตาม จำนวน Qubit) จึงเหมาะเป็นภาระงานตัวแทนสำหรับศึกษาขีดจำกัดฮาร์ดแวร์ขอบข่าย งานวิจัยนี้ พัฒนา C++ Quantum Circuit Micro-Simulator ขึ้นเอง แล้วนำไปทดสอบบนไมโครคอนโทรลเลอร์ 3 สถาปัตยกรรมที่ต่างกัน เพื่อสร้างกรอบการตัดสินใจว่าควรประมวลผลภายในเครื่อง (Local) หรือส่งไปประมวลผลภายนอก (Offload) เมื่อใด

วัตถุประสงค์

1. ออกแบบและพัฒนา C++ Quantum Circuit Micro-Simulator ที่ปรับให้เหมาะกับข้อจำกัด SRAM ของ MCU พร้อมตรวจสอบความถูกต้องเทียบกับ Qiskit Aer ด้วย Infidelity (1–F) และ Total Variation Distance (TVD)
2. ศึกษา Memory Wall เปรียบเทียบ Execution Latency ระหว่าง Local กับ Offload และหา Crossover Point ที่การ Offload คุ้มค่ากว่า พร้อมวิเคราะห์ผลของสถาปัตยกรรมที่มีและไม่มี FPU
3. สังเคราะห์ผลทั้งหมดเป็น Decision Framework รูปแบบ Flowchart และสมการ สำหรับวิศวกร เลือกวิธีประมวลผลบนระบบ Edge Computing

สมมติฐาน

1. Arduino Uno R3 (8-bit) จะถึง Memory Wall ที่จำนวน Qubit น้อยกว่า MCU 32-bit และต่ำกว่า ค่าทางทฤษฎี (2^n×8 Bytes) เนื่องจาก SRAM ต้องแบ่งให้ Stack และตัวแปรอื่นด้วย
2. เนื่องจาก Uno ไม่มี FPU Crossover Point จะเกิดขึ้นเร็วในช่วง Qubit ที่บอร์ดยังรันได้ ตรงข้ามกับ ESP32 ที่มี FPU และ Clock สูงกว่า ซึ่ง Crossover Point จะเลื่อนออกไปใกล้ Memory Wall
3. วงจร QFT ซึ่งมีจำนวนเกทมากกว่าวงจรพื้นฐาน (Hadamard, Bell/GHZ) จะคุ้มค่ากับการ Offload เร็วที่สุดในทุกบอร์ด

ตัวแปร

ตัวแปรต้น

สถาปัตยกรรม MCU (Uno / ESP8266 / ESP32), จำนวน Qubit, รูปแบบ การประมวลผล (Local / Offload)

ตัวแปรตาม

Execution Time และ End-to-End Latency (µs), Free Heap, ค่า Infidelity (1–F) และ TVD

ตัวแปรควบคุม

ชนิดวงจร (Hadamard ทุกคิวบิต, Bell/GHZ state, QFT), Baud Rate สูงสุดที่เสถียรของแต่ละบอร์ด, Compiler Optimization และเวอร์ชันเครื่องมือ

วิธีทำการทดลอง

Phase 1 — System Validation

เทียบ State Vector จาก C++ Simulator กับ Qiskit Aer ด้วยลำดับเกท เดียวกัน ทุกประการ วัด Infidelity (1–F) และ TVD ที่ n = 2, 3, 4, 6, 8, 10, 12 เกณฑ์ผ่าน Infidelity ≤ 10^{-3}

Phase 2 — Edge Benchmark

อัปโหลดและรันบน MCU ทั้ง 3 บอร์ด เพิ่มจำนวน Qubit ทีละ 1 จนถึง Memory Wall บันทึก Execution Time ด้วย Logic Analyzer 24 MHz และ Free Heap ซ้ำอย่างน้อย 30 รอบต่อจุดข้อมูล รายงานผลเป็น Median ± IQR

Phase 3 — Offload Benchmark

ส่งคำขอผ่านโปรโตคอลเฟรม Serial USB ให้ PC จำลองวงจรด้วย Qiskit Aer สดใหม่ทุกครั้งแล้วส่งผลกลับ วัด End-to-End Latency ซ้ำ 30 รอบต่อจุดข้อมูล ที่ Baud Rate สูงสุดที่ระบบยังทำงานได้เสถียร

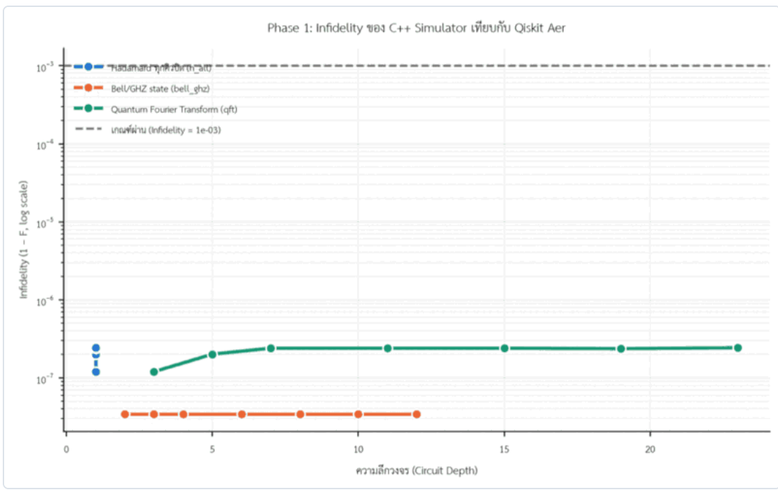
Phase 4 — Decision Framework

สังเคราะห์ผลจาก Phase 2–3 เป็น Flowchart 2 ขั้นตอน (ตรวจ Memory Wall ก่อน แล้วเทียบ Crossover Point) พร้อมสมการทำนายสำหรับบอร์ดที่ยังไม่ได้ทดสอบ

ฮาร์ดแวร์: Arduino Uno R3, ESP8266 V2, ESP32 DevKit V1, Logic Analyzer 24 MHz 8CH, MacBook Pro M1 • **ซอฟต์แวร์:** PlatformIO, Qiskit Aer, PySerial, NumPy, Matplotlib

ผลการทดลอง — Phase 1

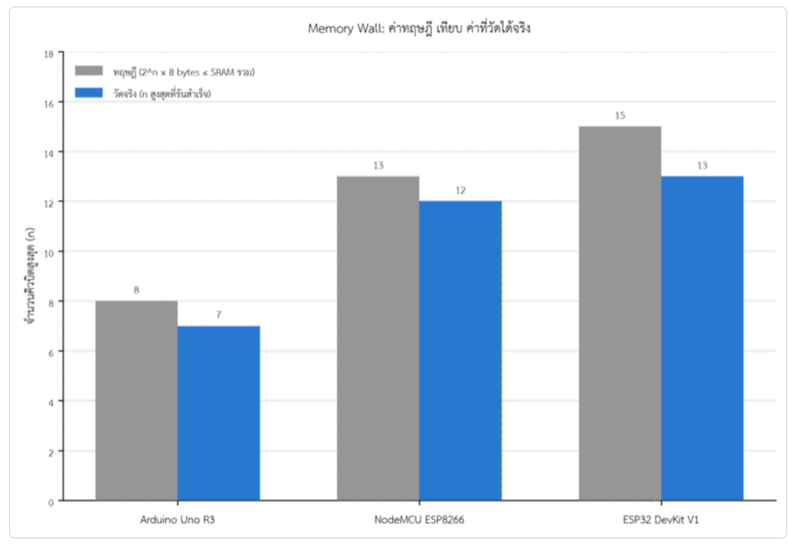
วงจร	Infidelity (1–F) สูงสุด	TVD สูงสุด	ผลตรวจ (7 ค่า n)
h_all	2.43×10^{-7}	1.22×10^{-7}	ผ่านทุกกรณี
bell_ghz	3.45×10^{-8}	1.72×10^{-8}	ผ่านทุกกรณี
qft	2.43×10^{-7}	1.22×10^{-7}	ผ่านทุกกรณี



ภาพที่ 1 Infidelity เทียบกับ Circuit Depth (log scale, 21 กรณีทดสอบ ครบ 3 วงจร × 7 ค่า n) — ค่าคลาดเคลื่อนอยู่ระดับ 10^{-7} ถึง 10^{-8} ต่ำกว่าเกณฑ์ผ่าน (10^{-3}) มากกว่า 4 อันดับ (order of magnitude) ยืนยันว่า C++ Simulator ที่พัฒนาขึ้นให้ผล ตรงกับ Qiskit Aer

ผลการทดลอง — Phase 2

บอร์ด	n ทฤษฎี	n วัดจริง	ส่วนต่าง
Arduino Uno R3	8	7	−1
NodeMCU ESP8266	13	12	−1
ESP32 DevKit V1	15	13	−2



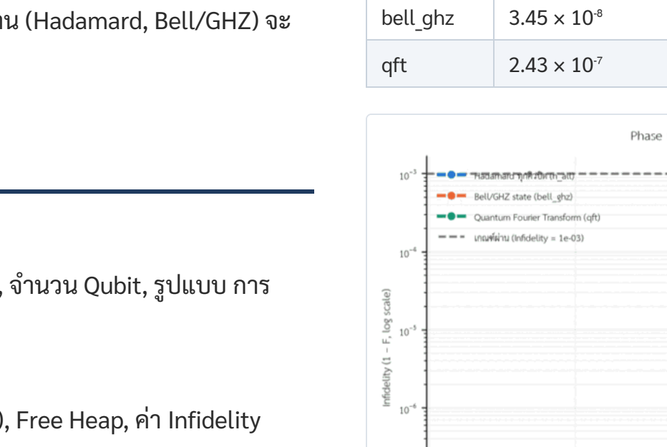
ภาพที่ 2 Memory Wall: n สูงสุดตามทฤษฎีเทียบค่าที่วัดได้จริง 3 บอร์ด — ทุกบอร์ดวัดได้ต่ำกว่าทฤษฎีเพราะต้องกันพื้นที่ให้ Stack/Global Variables โดย ESP32 เสียมากที่สุด (2 Qubit) เนื่องจากมี WiFi และ Bluetooth Stack พร้อมกัน สนับสนุน สมมติฐานข้อ 1

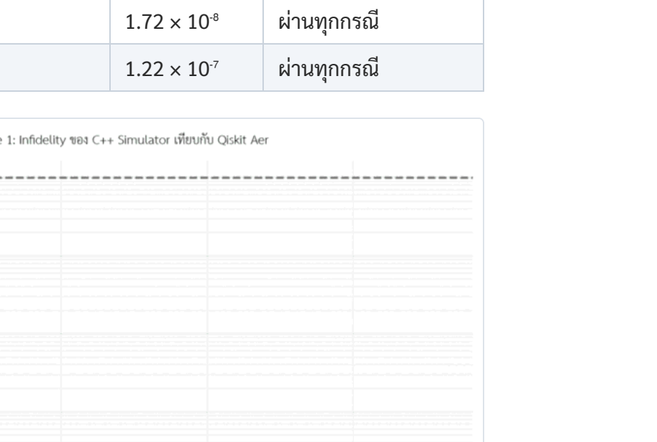
ผลการทดลอง — Phase 3 (Offload Benchmark)

วงจร	Uno	ESP8266	ESP32
h_all	5	8	13
bell_ghz	6	10	13
qft	4	7	12

ตัวเลข = Crossover Point (n ที่ Offload เริ่มเร็วกว่า Local)

ภาพที่ 3 Local Execution Time เทียบ Offload E2E Latency แยกกราฟต่อบอร์ด แต่ละกราฟแสดง 3 วงจร (วงกลมกลวง = Crossover Point) — Offload Latency เกือบคงที่ทุกบอร์ด (~1.6–14 ms) เพราะเฟรมข้อมูลมีขนาดคงที่ 4–6 bytes ทำให้ Crossover Point ขึ้นกับความเร็วคำนวณภายในเครื่องเป็นหลัก สนับสนุนสมมติฐานข้อ 2 และ 3





สรุปและอภิปรายผลการทดลอง

- **Offload Latency แทบคงที่** อยู่ในช่วง ~1.6–14 ms ไม่ขึ้นกับบอร์ด, Baud Rate หรือจำนวน Qubit เพราะเฟรมโปรโตคอลมีขนาดคงที่ (4–6 bytes) ต้นทุนส่วนใหญ่จึงเป็น fixed overhead ของ PC/OS/Driver
- **Crossover Point แปรผกผันกับความเร็วคำนวณของบอร์ดและความซับซ้อนของวงจร** — บอร์ดที่ไม่มี FPU (Uno) คุ้มกับ Offload ตั้งแต่ Qubit ต่ำ ในขณะที่ ESP32 ที่มี FPU ต้องรอถึง Qubit สูงใกล้ Memory Wall จึงเริ่มคุ้ม; วงจร QFT (เกทมากสุด) คุ้มกับ Offload เร็วที่สุดเสมอทุกบอร์ด
- **ทุกบอร์ดถึง Memory Wall ก่อนค่าทฤษฎี 1–2 Qubit** จากพื้นที่ที่ต้องกันไว้ให้ Stack และ Network/RTOS Stack — ยืนยันสมมติฐานข้อ 1 ทั้ง 3 บอร์ด
- ผลการทดลองสนับสนุนสมมติฐานทั้ง 3 ข้อที่ตั้งไว้ และนำไปสังเคราะห์เป็น Decision Framework 2 ขั้นตอนที่ใช้กับบอร์ดอื่นที่ยังไม่ได้ทดสอบได้ โดยมีสมการประมาณค่าดังนี้

สมการที่ 1 — Memory Wall

```
n_max = floor( log2( B / 8 ) )
```

B = largest contiguous free block ที่วัดได้จริง หากวัดไม่ได้ (บอร์ด static-allocation เช่น Uno) ประมาณด้วย **B ≈ SRAM รวม ÷ 2**

สมการที่ 2 — Crossover Point

```
n* ≈ n*_อ้างอิง + log2(clock ใหม่/clock อ้างอิง)
                + ตัวปรับสถาปัตยกรรม
```

ตัวปรับ: **+1 คิวบิต** เมื่อเปลี่ยน 8-bit → 32-bit (ไม่มี FPU ทั้งคู่), **+2 ถึง +3 คิวบิต** เมื่อเพิ่ม FPU ฮาร์ดแวร์ (สถาปัตยกรรม 32-bit เดิม)